

PROBABILISTIC ALGORITHMS IN FINITE FIELDS

By Michael Ben-Or

Department of Mathematics
Hebrew University
Jerusalem, Israel

Extended Abstract

1. Introduction

In this paper we deal with three computational problems in finite fields. The questions we deal with are the following:

- (i) Irreducible Polynomials - Finding an irreducible polynomial of degree n over $GF(q)$.
- (ii) Root Finding - Given a polynomial of degree n $f(x) \in GF(q)[x]$, we wish to find all the roots of $f(x) = 0$ in $GF(q)$.
- (iii) Factorization - Given a polynomial $f(x) \in GF(q)[x]$, we want to find the factorization $f = f_1 \dots f_r$ of f into its irreducible factors $f_i(x) \in GF(q)[x]$.

Algorithms for these problems were given by Rabin⁶, Berlekamp² and others (see [2] for references). In the first part of this paper we give an improved algorithm for finding irreducible polynomials. Our algorithm makes use of the fact that most polynomials of degree n over $GF(q)$ have low degree factors, making the irreducibility test more efficient on the average. This algorithm works as well even if we restrict our search to special types of irreducible polynomials such as $x^n + x + a$, $a \in GF(q)$.

In the second part of the paper we give an exact analysis of Rabin's root finding algorithm⁶. Using the deep results of A. Weil¹⁰, we are able to prove Rabin's conjecture that the probability of success in the random step of the algorithm is close to $1 - 1/2^{n-1}$ (where n is the degree of the polynomial). This improves Rabin's result to the effect that the probability is at least $1/2$.

The case of characteristic 2 is somewhat different. In this case the heuristic argument seems to suggest a similar conjecture but the actual state of affairs is different. Here we show that the best result is that the probability of success is at least $1 - \frac{1}{n}$.

Next we analyze the expected number of arithmetic operations in $GF(q)$ used by Rabin's algorithm for finding all the roots of an n degree polynomial. We show that finding all the roots is only $\log n$ times the expected number of operations for finding one of the roots by this method.

In the third part of the paper we deal with the factorization problem. We present a new probabilistic method based on Berlekamp's method. Using a probabilistic method we avoid the need of solution of linear equations occurring in Berlekamp's method, or the passage to computations in extensions of the field as in Rabin's method. This, when combined with some ideas from the root finding algorithm, leads to an algorithm which is simpler and faster than previous methods.

The complexity of the presented algorithms in terms of the expected number of arithmetical operations in $GF(q)$ are the following.

Denote $L(n) = \log n \cdot \log \log n$

- (i) An irreducible polynomial of degree n over $GF(q)$ can be found in $O(n^2 \log n L(n) \log q)$ operations.
- (ii) Finding all the roots of $f(x) \in GF(q)[x]$, $\deg f = n$, in $GF(q)$ can be done in $O(n \log n L(n) \log q)$ operations.
- (iii) Factoring $f(x) \in GF(q)[x]$, $\deg f = n$, can be done in $O(n^2 L(n) \log q)$ operations.

2. Irreducible Polynomial

In order to find a monic irreducible polynomial of degree n over $F = GF(q)$, Rabin⁶ has proposed a probabilistic algorithm based on the following result.

Theorem 1

Denote by $I(n)$ the number of monic irreducible polynomials of degree n over F . Then

$$I(n) = \frac{1}{n} \sum_{s \cdot t = n} \mu(s) q^t = \frac{1}{n} q^n + O(q^{\frac{n}{2}})$$

where $\mu(s)$ is the Mobius function.

The algorithm for finding an irreducible polynomial proceeds as follows:

Generate a monic random polynomial of degree n over $GF(q)$ and test it for irreducibility; continue until an irreducible polynomial is found. Theorem 1 ensures that the expected number of polynomials to be tried before an irreducible one is found is n .

As an improvement to Rabin's algorithm, we propose the replacement of the test for irreducibility by the following.

Lemma 1

A polynomial $g(x) \in GF(q)[x]$ of degree n is irreducible if and only if

$$(g(x), x^{q^k} - x) = 1 \text{ for } k=1, \dots, \lfloor \frac{n}{2} \rfloor.$$

Lemma 1 suggests the following algorithm.

Algorithm A

1. Generate a monic random polynomial $g(x)$ of degree n over $GF(q)$;
2. $u(x) \leftarrow x$;
3. for $k \leftarrow 1$ to $\lfloor \frac{n}{2} \rfloor$ do
4. $u(x) \leftarrow u(x)^q \bmod g(x)$;
5. if $(g(x), u(x) - x) \neq 1$ then go to 1 fi;
6. od;

Let $g(x) \in GF(q)[x]$. Denote by $m(g)$ the minimal degree of an irreducible factor of $g(x)$ in $GF(q)[x]$. Then the number of operations used by algorithm A to test $g(x)$ is $O(m(g) \cdot nL(n) \log q)$. The test for irreducibility in algorithm A is less efficient than the test suggested by Rabin⁶ in case the polynomial $g(x)$ happens to be irreducible. However the main idea behind algorithm A is the following.

Theorem 2

The expected value of $m(g)$ for a random polynomial $g = g(x) \in GF(q)[x]$ of degree n is $O(\log n)$.

Proof

One can show by a counting argument that the probability that a random polynomial of degree n will have an irreducible factor of degree k is the same as the probability that a random permutation on n symbols will have a cycle of length k . So

our claim follows from the fact that the expected length of the shortest cycle in a random permutation is $O(\log n)$ ⁹.

Thus we have:

Theorem 3

The expected number of operations used to test a random polynomial of degree n over $GF(q)$ for irreducibility is $O(n \log n L(n) \log q)$, and the expected number of operations for finding an irreducible polynomial of degree n is

$$O(n^2 \log n L(n) \log q).$$

Irreducible polynomials of degree n over $Z_p = GF(p)$, p prime, are used to represent the arithmetic of $GF(p^n)$. Given such a polynomial $g(x)$ we can represent $GF(p^n)$ as the set of polynomials of degree less than n over Z_p , with addition and multiplication mod $g(x)$. In order to make residue computation mod $g(x)$ easier one looks for special types of irreducible polynomials such as $g(x) = x^n + x + a$, $a \in Z_p$. Using the density theorem of A. Weil, Ree⁷ and Leonard⁵ proved that $x^n + x + a$ factors like the general polynomial. Thus one can show

Theorem 4

- (i) [7] the number of polynomials

$$x^n + x + a \quad 0 \leq a < p$$

which are irreducible in $Z_p[x]$ is $\sim \frac{p}{n}$

- (ii) [5,9] The expected value of

$$m(x^n + x + a) \text{ is } O(\log n)$$

Corollary 1

The expected number of operations used to find an irreducible polynomial of the form $x^n + x + a$ is $O(n^2 \log n L(n) \log q)$.

3. Root Finding in $GF(q)$

Given a polynomial $f(x) \in GF(q)[x]$ of degree n we wish to find all the roots $a \in F = GF(q)$ of $f(x) = 0$.

Assume that q is odd. We shall treat later the important case $q = 2^m$.

Form the g.c.d.

$$f_1(x) = (f(x), x^q - x).$$

If $f_1(x) = 1$ then $f(x)$ has no roots in F . In general

$$f_1(x) = (x - a_1) \dots (x - a_k) \quad k \leq n$$

where the a_i are all the pairwise distinct roots of $f(x) = 0$ in F . In [6] Rabin proposed a randomized algorithm for finding the roots of

$f_1(x) = 0$, which we describe in the following recursive way.

Algorithm B

Procedure ROOTS ($f_1(x)$)

1. if $\deg f_1(x) = 1$ (i.e. $f_1(x) = x-a$) then
return (a);
2. repeat
3. choose $\delta \in F$ randomly
4. $f_2(x) \leftarrow (f_1(x), (x+\delta)^{(q-1)/2-1})$
5. until $0 < \deg f_2 < \deg f_1$
6. return (ROOTS ($f_2(x)$) $\cup$ ROOTS ($f_1(x)/f_2(x)$)).

In [6] Rabin proves that the probability that on line 5 we will have $0 < \deg f_2 < \deg f_1$ is at least $1/2$, and conjectures that it is close to $1 - 1/2^{k-1}$, where $k = \deg f_1$. We prove this conjecture and give a full analysis of the expected number of operations used by the above algorithm to find all the roots of $f_1(x) = 0$.

Let χ be the quadratic character of F , i.e., $\chi(a) = 1$ if a is a non zero square in F , $\chi(0) = 0$, and $\chi(a) = -1$ otherwise. The roots of $x^{(q-1)/2} - 1$ are all the squares in F thus the common roots of $f_1(x)$ and $(x+\delta)^{(q-1)/2} - 1$ are those a_i , $f_1(a_i) = 0$ for which $\chi(a_i + \delta) = 1$.

Lemma 2

Let $a_1, \dots, a_k$ be pairwise distinct elements of F and let $\epsilon_\ell \in \{1, -1\}$ $\ell=1, \dots, k$. Then $N = |\{\delta \in F \mid \chi(a_\ell + \delta) = \epsilon_\ell, \ell=1, \dots, k\}| = \frac{q}{2} + O(q^{1/2})$.

Proof

Let $d \in F$ be a fixed non square, i.e. $\chi(d) = -1$. Let M be the number of solutions

$(x, y_1, \dots, y_k) \in F^{k+1}$ of the system of equations $y_1^2 = d_1(x+a_1), \dots, y_k^2 = d_k(x+a_k)$

where d_ℓ equals d or 1 according as to whether $\epsilon_\ell = -1$ or 1 .

For any $w \in F$ the number of solutions to the equation $y^2 = w$ is given by $1 + \chi(w)$. Hence

$$\begin{aligned} M &= \sum_{x \in F} \prod_{\ell=1}^k [1 + \chi(d_\ell x + d_\ell a_\ell)] = \\ &= \sum_{x \in F} \sum_{0 \leq i_1, \dots, i_k \leq 1} \prod_{\ell=1}^k \chi((d_\ell x + d_\ell a_\ell)^{i_\ell}) = \\ &= \sum_{0 \leq i_1, \dots, i_k \leq 1} \sum_{x \in F} \chi(\prod_{\ell=1}^k (d_\ell x + d_\ell a_\ell)^{i_\ell}) \end{aligned}$$

The main term is for $i_1 = \dots = i_k = 0$ and it equals

q. The other summands are character sums

$$\sum_{x \in F} \chi(g(x))$$

where $g(x)$ is a polynomial of degree ℓ , $1 \leq \ell \leq k$, with ℓ distinct roots, by [10, 11 Theorem 2C']

$$\left| \sum_{x \in F} \chi(g(x)) \right| < (\ell-1)q^{1/2}.$$

$$\text{Thus } |M-q| < \sum_{\ell=1}^k \binom{k}{\ell} (\ell-1)q^{1/2} < (2^{k-1}-1)kq^{1/2}.$$

If M' is the number of solutions $(x, y_1, \dots, y_k)$ with all y_ℓ non zero then $|M-M'| < k \cdot 2^{k-1}$ and $M' = 2^k \cdot N$. Thus $|N-q/2^k| < \frac{k}{2}(q^{1/2}+1)$.

Corollary 2

The probability that $0 < \deg f_2 < \deg f_1$ on line 5 of the root finding procedure is at least

$$1 - \frac{1}{2^{k-1}} + O\left(\frac{1}{\sqrt{q}}\right)$$

where $k = \deg f_1$.

Proof

$\deg f_2 = 0$ only if $\chi(a_\ell + \delta) = -1$ for $\ell=1, \dots, k$ and $\deg f_2 = \deg f_1$ if $\chi(a_\ell + \delta) = 1$ for all ℓ . The probability of each of these events is by the lemma $1/2^k + O(1/\sqrt{q})$. Thus the probability that both will not occur is at least $1 - 1/2^{k-1} + O(q^{-1/2})$. If q is large we can prove more.

Theorem 5

The expected number of operations used by algorithm B to find all the roots of $f(x)=0$ in F , $\deg f = n$, is $O(n \log n L(n) \log q)$.

Proof

The computation of $f_1(x)$ and $f_2(x)$ can be done by $O(nL(n) \log q)$ operations. By lemma 2 the probability that $\deg f_2 = \ell$ on line 4 of the algorithm is $\binom{k}{\ell} 2^{-k}$ for $\ell=0, 1, \dots, k$, $k = \deg f_1$. Thus if $T(k)$ denotes the expected number of operations used to find all the roots of $f_1(x)$ we have

$$T(k) \leq ckL(k) \log q + \frac{1}{2^k} \sum_{\ell=0}^k \binom{k}{\ell} [T(\ell) + T(k-\ell)]$$

and by induction $T(k) \leq 2ck \log k L(k) \log q$.

The assumption "q large" can be removed by the method of the next section.

The case of characteristic 2, $q = 2^m$

In this case we cannot use the polynomial $x^{(q-1)/2} - 1$ on line 4 of algorithm B. However a small modification does work [6]. We use the trace function $\text{Tr}(x)$ of the field $F = \text{GF}(2^m)$ to its prime field $\text{GF}(2) = \mathbb{Z}_2$, i.e. the polynomial

$$\text{Tr}(x) = x + x^2 + \dots + x^{2^{m-1}}$$

and modify line 4 of the algorithm to be

$$f_2(x) = (f_1(x), \text{Tr}(\delta \cdot x))$$

The roots of $f_2(x)$ are those $a \in F$ such that $f_1(a)=0$ and $\text{Tr}(\delta a)=0$. In order to find the probability distribution of the degree $f_2(x)$ we note the following. Fix $\delta \in F$, then $\text{Tr}(\delta x)$ is a linear functional on F to Z_2 . As δ ranges over F , $\text{Tr}(\delta x)$ ranges over all the linear functionals from F to Z_2 .

Given $a_1, \dots, a_k \in F$ we define random variables $X_\ell = \varphi(a_\ell)$ where φ is a randomly selected linear functional on F , and define $Y = X_1 + \dots + X_k$. We are interested in the probability distribution of Y .

We note without proofs the following easy lemmata.

Lemma 3:
The expectation of X_ℓ , $E(X_\ell)$ is $\frac{1}{2}$ if $a_\ell \neq 0$ and 0 if $a_\ell = 0$.

Lemma 4:
If $a_\ell \neq 0$ then $E(Y) = \frac{k}{2}$.

Lemma 5:
If $a_1, \dots, a_k$ are linearly independent over Z_2 then $X_1, \dots, X_k$ are mutually independent random variables, so Y has the binomial distribution.

Lemma 6:
If $a_i \neq 0$ and $a_i \neq a_j$ if $i \neq j$, then the variance of Y , $V(Y)$ equals $k/4$.

Lemma 7:
If $a_i \neq 0$ and $a_i \neq a_j$ if $i \neq j$, then $P(|Y - \frac{k}{2}| > \sqrt{k}) < \frac{1}{4}$
and $P(|Y - \frac{k}{2}| \leq \frac{k}{2}) \leq \frac{1}{k}$.

As a corollary we get.

Corollary 3:

The probability that $0 < \deg f_2 < \deg f_1$ on line 5 of the root finding algorithm is at least $1 - \frac{1}{k}$, where $k = \deg f_1$.

Lemma 7 suggests the following modification to our algorithm. We keep on drawing random δ till $f_2(x)$ is a non-trivial factor of $f_1(x)$ and the degree of $f_2(x)$ is in the range

$[\frac{k}{2} - \sqrt{k}, \frac{k}{2} + \sqrt{k}]$. Doing so one can show that the expected number of operations used by this modified algorithm is $O(n \log n L(n) \log q)$. Now it is easy to see that the original algorithm uses less operations than the modified one. Thus we have

Theorem 7:

The expected number of operations used by the root finding algorithm for finding all the roots of an n -degree polynomial is

$$O(n \log n L(n) \log q).$$

Note 1: One can use the method of this section with minor changes to prove theorem 5 even for small q . However without lemma 2 one cannot

prove corollary 2 and the best result will be the result of corollary 3.

Note 2: One cannot sharpen the result of corollary 3 because if $f(x) = (x-a_1) \dots (x-a_n)$ where the a_ℓ are all the elements of a linear subspace of F over Z_2 , then $f(x) = f_1(x)$, $\deg f_1 = n$ and the probability of not factoring $f_1(x)$ is exactly $\frac{1}{n}$.

5. Factorization of Polynomials

Let $f(x) \in F[x]$, $F = GF(q)$ be a polynomial of degree n which we want to factor into its irreducible factors. We may assume (see [2] [6]) that $f(x)$ has no repeated factor. We now factor $f(x) = g_1(x) \dots g_d(x)$, where $g_d(x)$ is the product of the irreducible factors of $f(x)$ of degree d , by the following well known iteration: Define $r_0(x) = x$, $f_0(x) = f(x)$ and then for $d=1, \dots, n$

$$r_d(x) = r_{d-1}(x)^q \bmod f_{d-1}(x), \deg r_d < \deg f_{d-1}$$

$$g_d(x) = (f_{d-1}(x), r_d(x) - x)$$

$$f_d(x) = f_{d-1}(x) / g_d(x).$$

This factorization can be done in $O(n^2 L(n) \log q)$ operations. Thus we have reduced the problem to the factorization of polynomials with all factors of the same degree.

Let $g_d(x) = g(x) = h_1(x) \dots h_k(x)$, $k \cdot d = \deg g_d$, where the $h_i(x)$ are distinct irreducible polynomials of degree d . To separate the factors of $g(x)$ note [2] that any polynomial $s(x)$ such that $s(x)^q - s(x) = 0 \bmod g(x)$, $0 < \deg s < d$

gives us a non-trivial factorization of $g(x)$. Namely

$$g(x) = \prod_{\alpha \in GF(q)} (g(x), s(x) - \alpha)$$

We call such a polynomial a separating polynomial for $g(x)$. To find such polynomials we proceed in the following new probabilistic way. Consider the ring $R = F[x]/(g(x))$ and the map $\text{Tr}: R \rightarrow R$ given by

$$\text{Tr}(y) = y + y^q + \dots + y^{q^{d-1}}$$

Theorem 8

- (i) Tr is a linear map on R_g (as a linear space over F) of rank k .
- (ii) $\text{Tr}(y)^q = \text{Tr}(y)$.
- (iii) For a randomly chosen $y \in R$, $\text{Tr}(y)$ is a separating polynomial for $g(x)$ with probability $1 - 1/q^{k-1}$.
- (iv) With probability at least $1 - k(k-1)/2q$, $\text{Tr}(y)$ will separate all the factors of $g(x)$.

Proof

By the Chinese Remainder Theorem R_g is isomorphic to the direct sum

$R_{h_1} + \dots + R_{h_k}$ under the map $y \rightarrow (y_1, \dots, y_k)$ with $y_k \equiv y \pmod{h_k}$. Thus $\text{Tr}(y) = \text{Tr}(y_1, \dots, y_k) = (\text{Tr}(y_1), \dots, \text{Tr}(y_k)) = (a_1, \dots, a_n)$. Since the h_i are irreducible of degree d each R_{h_i} is a field and Tr is the trace function of this field to $F = \text{GF}(q)$. Thus $(a_1, \dots, a_k) \in F^k$ and by choosing $y_1, \dots, y_k$ we can get any vector $(a_1, \dots, a_k) \in F^k$. This proves (i). $\text{Tr}(y) = (a_1, \dots, a_k)$ will separate h_i from h_j if $a_i \neq a_j$. Thus $\text{Tr}(y)$ is a separating polynomial unless $a_1 = \dots = a_k$, and probability of this event is $1/q^{k-1}$. $\text{Tr}(y)$ will separate all the h_i if $i \neq j \Rightarrow a_i \neq a_j$ and this has probability at least $1 - k(k-1)/2q$.

After finding a separating polynomial we use the same random shift as in the root finding algorithm to factor $g(x)$.

Algorithm C

(separating factors of equal degree)

Procedure factor ($g(x)$);
 $g(x) = h_1 \dots h_k$, h_i irreducible $\deg h_i = d$

1. choose a random polynomial y , $\deg y < \deg g$;
2. $s(x) \leftarrow \text{Tr}(y)$;
3. if $\deg s = 0$ then go to 1;
4. call Separate ($g(x)$, $s(x)$);

Procedure Separate ($g(x)$, $s(x)$);

5. if $\deg g(x) = d$ then return ($g(x)$);
6. $s(x) \leftarrow s(x) \pmod{g(x)}$;
7. if $\deg s = 0$ then call Factor ($g(x)$);
8. choose $\delta \in F$ randomly;
9. $g_1(x) \leftarrow (g(x), (s(x) + \delta)^{(q-1)/2 - 1})$;
10. if $\deg g_1 = 0$ or $\deg g_1 = \deg g$ then go to 8;
11. return (Separate (g_1) U Separate (g/g_1));

[If $q = 2^m$ we replace line 9 by $g_1(x) \leftarrow (g(x), y^{2^{m-1}} + \delta \cdot s(x))$ where $T(y) = y + y^2 + \dots + y^{2^{m-1}}$.]

By theorem 8 the expected number of times we will compute a separating polynomial is close to 1 (if q is large), thus by the same analysis of the root finding algorithm we have

Theorem 9

The expected number of operations used to factor $g(x) = h_1 \dots h_k$ where the h_i are distinct irreducible polynomials of degree d , $\deg g = \ell = d \cdot k$ is $O((d + \log k) \ell L(\ell) \log q)$.

Theorem 10

The expected number of operations used to factor an n -degree polynomial is $O(n^2 L(n) \log q)$.

Note 3: In [3] Calmet and Loos claim that using Berlekamp's $O(n^3)$ matrix method is more efficient than the method used above to factor $f(x)$ to the product $f(x) = g_1(x) \dots g_n(x)$. However their step count analysis is erroneous because they used classical polynomial arithmetic instead of the fast algorithms based on the FFT used in [6] and here.

Independently Cantor and Zassenhaus⁴ used a similar idea to separate factors of equal degree in the factorization problem. Using their algorithm one can factor a product of k factors of degree d by $O(d \cdot \log k \ell L(\ell) \log q)$ where $\ell = d \cdot k$. Thus our algorithm is slightly better than theirs.

In the case of characteristic 2, $q = 2^m$ the algorithm in [4] uses a primitive third root of unity. Thus if $q \equiv 2 \pmod{3}$, they factor the polynomial in the quadratic extension of the field and then combine conjugate factors. In our algorithm all the computations are carried within the field without passing to any extension of the field.

References

- [1] A.V. Aho, J.E. Hopcroft and J.D. Ullman, The Design and Analysis of Algorithms, Addison-Wesley, Reading Mass. 1974.
- [2] E.R. Berlekamp, Factoring polynomials over Large Finite Fields, Math. Comput. 24(1970) pp. 713-735.
- [3] J. Calmet and R. Loos, An Improvement of Rabin's Probabilistic Algorithm for Generating Irreducible Polynomials over $\text{GF}(p)$, Information Proc. Lett. 11(1980), pp. 94-95.
- [4] D.G. Cantor and H. Zassenhaus, On Algorithms For Factoring Polynomials over Finite Fields. To appear.
- [5] P.A. Leonard, Factorization of General Polynomials. J. of Number Theory 6(1974) pp. 335-338.
- [6] M.O. Rabin, Probabilistic Algorithms in Finite Fields, SIAM J. of Comput. 9(1980) pp. 273-280.
- [7] R. Ree, Proof of a Conjecture of S. Chowla, J. Number Theory 3(1971), pp. 210-212.
- [8] A. Schonhage, Schulle Multiplikation von Polynomen Uber Korpen der Charakteristic 2, Acta Informatica 7(1977), pp. 395-398.
- [9] L.A. Shepp and S.P. Lloyd, Ordered cycle lengths in a random permutation, Trans. A.M.S. 121(1966), pp. 340-357.
- [10] A. Weil, Sur Les Courbes Algebriques et Les Varietes qui s'en deduisent, Herman Paris (1948).
- [11] W.M. Schmidt, Equations over finite fields. An Elementary Approach, Springer Verlag, Lecture Notes in Math., 536(1976).